

PYTHON LEARNING FOR DATA ANALYSIS: DAY 04

Dispersion Post-Mortem: Range, Variance, Standard Deviation & Coefficient of Variation

1. WHY DISPERSION MATTERS? (BUSINESS PERSPECTIVE)

Data Analysis mein sirf Average (Mean) nikalna kaafi nahi hota. Do alag business scenarios ka average same ho sakta hai, lekin unka behavior bilkul alag ho sakta hai. Isi bikhraav ko samajhne ke liye hum Dispersion seekhte hain.

Real Business Scenario: Maaniye do customers har mahine gold coins kharidte hain. Dono ka Average (Mean) 15 Grams hai.

- **Customer A:** [15, 15, 15, 15] → Har mahine barabar 15 gram leta hai. Ekdum steady aur predictable business!
- **Customer B:** [5, 25, 5, 25] → Kisi mahine achanak 5 gram aur kisi mahine seedhe 25 gram. Iska data bikhra hua (scattered) hai.

Dispersion humein batata hai ki data mein risk aur volatility kitni hai.

2. RANGE: MANUAL LOGIC VS BUILT-IN METHODS

Range data ke sabse bade number (Maximum) aur sabse chhote number (Minimum) ke beech ka farq hota hai.

A. Manual Loop Logic (No Built-in Functions):

```
# Karigar B ka gold chain production data (in grams)
data = [5, 25, 12, 18, 9]

# Step 1: Starting variables ko list ke pehle element par set karna
sabse_bada = data[0]    # Shuruat mein 5 ko sabse bada mana
sabse_chhota = data[0]  # Shuruat mein 5 ko hi sabse chhota mana

# Step 2: Loop chala kar ek-ek number ko check aur compare karna
for x in data:
    if x > sabse_bada:
        sabse_bada = x    # Agar koi bada number mile, toh use update karein

    if x < sabse_chhota:
        sabse_chhota = x  # Agar koi chhota number mile, toh use update karein

# Step 3: Range nikalna (Maximum aur Minimum ka difference)
range_value = sabse_bada - sabse_chhota

print(f"Sabse Badi Value (Max): {sabse_bada}")
print(f"Sabse Chhoti Value (Min): {sabse_chhota}")
print(f"Data ki Range hai: {range_value}")
```

Output:

Sabse Badi Value (Max): 25

Sabse Chhoti Value (Min): 5

Data ki Range hai: 20

B. Built-in max() & min() Solution:

```
data = [5, 25, 12, 18, 9]

sabse_bada = max(data)    # direct 25 dhoondh lega
sabse_chhota = min(data)  # direct 5 dhoondh lega
range_value = sabse_bada - sabse_chhota

print(f"Data ki Range hai: {range_value}")
```

3. VARIANCE: MANUAL LOGIC & TYPES

Variance humein batata hai ki har ek number hamare normal average (Mean) se kitna door bhaag raha hai.

A. Pure Manual Logic Code:

```
data = [10, 20, 15]

# STEP 1: Mean (Average) nikalna
total_sum = sum(data)
total_elements = len(data)
mean = total_sum / total_elements

# STEP 2: Har element ka Mean se farq (deviation) nikal kar uska Square karna
squares_list = []
for x in data:
    farq = x - mean          # Mean se dhoori nikali
    square_farq = farq ** 2  # Minus sign khatam karne ke liye square kiya
    squares_list.append(square_farq)

# STEP 3: Un saare squared values ka average (Variance) nikalna
variance = sum(squares_list) / len(squares_list)

print(f"1. Data ka Mean hai: {mean}")
print(f"2. Squared Deviations ki list: {squares_list}")
print(f"3. Final Manual Variance hai: {variance:.2f}")
```

Output:

1. Data ka Mean hai: 15.0
2. Squared Deviations ki list: [25.0, 25.0, 0.0]
3. Final Manual Variance hai: 16.67

4. POPULATION VARIANCE VS SAMPLE VARIANCE

Core Concept Breakdown:

Population Variance (pvariance): Jab hamare paas 100% data maujood ho. Formula mein niche total items (n) se divide karte hain. Exact aur strict answer milta hai.

Sample Variance (variance): Jab data kisi bade group ka ek chhota tukda (sample) ho. Safe margin rakhne ke liye formula mein ($n - 1$) se divide karte hain taaki risk underestimate na ho.

Library Code Shortcut:

```
import statistics

data = [10, 20, 15]

# TYPE 1: POPULATION VARIANCE (pvariance) -> Divides by 'n'
pop_variance = statistics.pvariance(data)

# TYPE 2: SAMPLE VARIANCE (variance) -> Divides by 'n - 1'
samp_variance = statistics.variance(data)

print(f"1. Population Variance (Divided by 3): {pop_variance:.2f}")
print(f"2. Sample Variance (Divided by 3 - 1 = 2): {samp_variance:.2f}")
```

Output:

1. Population Variance (Divided by 3): 16.67
2. Sample Variance (Divided by 3 - 1 = 2): 25.00

5. STANDARD DEVIATION (SD): REAL ROUTINE RISK

Variance mein unit square ho jati hai. Us unit ko wapas normal karne ke liye hum Variance ka **Square Root** nikalte hain.

A. Manual Setup:

```
import math

data = [10, 20, 15]
mean = sum(data) / len(data)
squares_list = [(x - mean) ** 2 for x in data]
variance = sum(squares_list) / len(data)

# SD nikalna (Variance ka Square Root)
std_deviation = math.sqrt(variance)
print(f"Final Manual Standard Deviation (SD): {std_deviation:.2f}")
```

Output: Final Manual Standard Deviation (SD): 4.08

B. Library Shortcut:

```
import statistics

data = [10, 20, 15]
pop_sd = statistics.pstdev(data) # Population SD
samp_sd = statistics.stdev(data) # Sample SD

print(f"1. Library Population SD: {pop_sd:.2f}")
print(f"2. Library Sample SD: {samp_sd:.2f}")
```

Output:

1. Library Population SD: 4.08
2. Library Sample SD: 5.00

6. COEFFICIENT OF VARIATION (CV): SACHA MUQABLA

CV do alag-alag level ke datasets ka sachha relative risk percentage (%) mein nikalta hai.

$$CV = (Standard\ Deviation / Mean) * 100$$

```
import statistics

gold_prices = [4950, 5050, 5000]
silver_prices = [90, 110, 100]

# Gold Analysis
gold_mean = statistics.mean(gold_prices)
gold_sd = statistics.pstdev(gold_prices)
gold_cv = (gold_sd / gold_mean) * 100

# Silver Analysis
silver_mean = statistics.mean(silver_prices)
silver_sd = statistics.pstdev(silver_prices)
silver_cv = (silver_sd / silver_mean) * 100

print(f"Gold CV (Risk %): {gold_cv:.2f}%")
print(f"Silver CV (Risk %): {silver_cv:.2f}%")
```

Output:

Gold CV (Risk %): 0.82%

Silver CV (Risk %): 8.16%

Conclusion: Silver market zyada volatile (risky) hai!

Day 04 Summary Rule: Mean target batata hai, Range extreme bounds batata hai, SD actual routine risk (bhatkaav) batata hai, aur CV percentage mein badal kar do alag metrics ka sachha muqabla karwata hai!